

plication hangs. It doesn't define the features, but this is where the features will plug-in.

Let's use a modern kitchen appliance as the example. Consider a high-end countertop convection oven (Fig. 1) with digital controls and an LED display.

The high-level structure of the product dictates a heating system (electric heating elements and controls), a fan system (the convection part), a power supply, and a user interface with touch-control buttons and LED display. Looking at the embedded system design, we have a power system (managing the safety, perhaps watching for power loss), an oven system with sub-systems controlling the heating elements, a fan using temperature feedback, and a user interface for user input and display, which may additionally have menu/recipe storage and presets. This structural definition is pretty stable, with a low risk of change.

At high risk of change at this point are things like whether the design consists of one super-MCU doing everything with all possible hardware inside the package (a fantasy) or several MCUs and off-the-shelf peripherals whose details are highly subject to change. The structure must transcend the specific hardware device selections and concentrate on the application. Whether the design has one or several MCUs, there will be one main MCU and that is our focus as we define the structure.

The structure definition focuses on the end-application, as shown in Fig. 2. On the left of the diagram are the inputs (blue blocks): temperatures, user interaction and saved recipes. On the right are the outputs (red blocks): the heating element, the convection fan and the display to the user. Tying it all together, in the centre, is the control logic block where the central decision-making takes place. Because this design will still need to be allocated amongst the lower-level sub-systems and devices, there is no need to know or decide how the arrows in this diagram are implemented. That doesn't

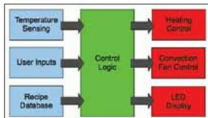


Fig. 2: Whole oven block diagram

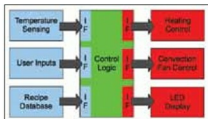


Fig. 3: Control logic with interfaces

need to happen until after step 4, and even if it is known and rock-solid at this point, it doesn't change what happens in steps 2 through 4.

Step 2: Establish the communications

Each of the blocks in Fig. 3, at this point, can be considered a distinct sub-system. The communications between them is through the interface definition. Nailing down the interfaces is critical to capturing the essence or structure of the product and allowing the requirements in each sub-system to evolve independently (that is, limiting any impact on the project schedule and otherwise). The focus of this step is the control logic (the green block)—the heart of the product.

Communication refers to the information flowing into and out of the control logic block, and this may involve messaging supported by a real-time operating system (RTOS), a physical communications interface like FC or SPI, or simply function calls. It is still too early to know all of these details and the goal is to set the interface so that the control logic block never has to know which method has been used. That way the core application (as implemented in the control logic block) proceeds unaffected by fluctuations in other blocks.

To establish the communications,

identify from the point of view of the control logic what is needed from each input or output block. Consider temperature sensing, for instance. From this block, the control logic needs to know the current temperature (assume for the example that there is a single temperature zone and therefore only one temperature to worry about). The format of the temperature is an important aspect of this interface, as changes to this greatly impact the implementation of the control logic. Therefore set the format and define the function that the control logic block will use: perhaps a C-language function prototyped as 'int getTemperature(void);' where the result of the temperature is a signed fixed-point integer representing the temperature times 10 (i.e., 353.7 degrees would be returned as 3537). Units would also be important and either these would be defined as part of the getTemperature interface description or two different functions would be defined—one for temperature in Fahrenheit and the other in degrees Celsius (getTemperatureF() and getTemperatureC()).

Continue defining interfaces for all the blocks (depicted in Fig. 3 as blue and red 'I/F' blocks contained inside the green 'control logic' block).

Step 3: Build-in the timing

Timing is a critical design requirement and as such it is subject to modification as the project proceeds. At this stage, the most important thing to determine is how the different blocks will be timed. Will the temperature block asynchronously update and the control block just access the most recent value, or will the completion of a new update interrupt the control logic and start a new frame? These types of decisions need to be made and depend upon a couple of factors, such as how fast something happens and how fast a response is expected.

The simplest design is for each block to operate independently, providing the most current data to the control logic (in the case of inputs)